Optimizing Local Large Language Model Inference: A Survey of Methods and an Empirical Case for Measurement-Based Placement

A review of quantization, decoding-algorithm, KV-cache, offloading, batching and multi-model coordination techniques for running large language models on resource-constrained consumer hardware, and a measured evaluation of a runtime placement layer against them.

Neel
Ramer project — d:\ramer
24 September 2026

Abstract

Running a large language model on a single consumer GPU is now routine, but doing it *well* is not. The research literature offers a deep and fast-moving set of levers — post-training quantization, speculative decoding, KV-cache paging and compression, weight offloading across GPU, host RAM and flash, continuous and chunked-prefill batching, and cross-model memory coordination — yet most of this work is designed, evaluated and tuned for datacenter conditions: many concurrent requests, throughput as the objective, and GPUs whose memory is measured in tens of gigabytes. This survey organizes that literature into six families, states for each what it actually buys, what it costs, and whether it transfers to an 8 GB desktop card driving a single interactive agent. We then identify a gap the six families leave open. Every production engine must decide how to split a model between GPU and host memory, and every one of them decides this with a *static estimate* that is deliberately conservative to avoid out-of-memory failures. We report measurements from one real machine (RTX 5060, 8 GB VRAM) showing that this conservatism left between 6.7% and 402% of achievable decode throughput unclaimed across three models, that the correct split is a function of context length and not of the model alone, and that crossing the physical VRAM boundary triggers a driver-level paging cliff which can make GPU offload slower than using no GPU at all. We describe Ramer, a runtime layer that replaces the estimate with a cached empirical probe, keys each placement to the context window it was measured at, breaks throughput ties in favour of VRAM headroom, re-validates at run time, and arbitrates model residency across concurrent agents. Against the engine's own default it improves decode throughput by 61%, 6.7% and 402% on three models, and cuts multi-agent wall time from a 210.6 s to a 117.6 s median. We also report, at equal prominence, the boundaries of the result: the benefit is proportional to the share of a workload that is genuinely local decoding, it disappears on tool-latency-bound tasks, it cannot rescue an inadequate model, and an upstream auto-fitting facility landed in llama.cpp in December 2025 that attacks the same estimation problem from inside the engine. The contribution we defend is therefore narrow and specific: *measurement beats estimation for placement on memory-constrained single-GPU machines, and the measurement must be keyed to context length*.

Keywords: local LLM inference, edge AI, GPU memory management, model offloading, quantization, KV cache, inference scheduling, autotuning.

Contents

1. Introduction **4**

1.1 Scope and the regime under study . . . 4

1.2 Contributions . . . 4

1.3 Method of the survey . . . 5

2. Background: where the time and the memory go **5**

3. A taxonomy of optimization methods **5**

4. Review of the families **7**

4.1 F1: model compression . . . 7

4.2 F2: decoding-algorithm acceleration . . . 7

4.3 F3: KV-cache management . . . 8

4.4 F4: weight placement and offloading . . . 8

4.5 F5: batching and request scheduling . . . 9

4.6 F6: multi-model resource coordination . . . 9

4.7 Cross-cutting: configuration auto-tuning . . . 10

5. How the families compose, and what a local stack actually uses **10**

6. Five gaps in the constrained single-machine regime **11**

6.1 Placement is estimated, never measured . . . 11

6.2 Placement validity is coupled to context length, and this is ignored . . . 11

6.3 The oversubscription cliff is undefended . . . 11

6.4 Single-machine multi-agent contention is managed, but not by policy . . . 11

6.5 Agentic workloads are not benchmark workloads . . . 11

7. Our method: Ramer **12**

7.1 Design principles . . . 12

7.2 The placement algorithm . . . 12

7.3 Two rules that were learned, not designed . . . 12

7.4 Multi-agent scheduling . . . 13

7.5 Integration . . . 13

8. Empirical evaluation **13**

8.1 Setup and honesty protocol . . . 13

8.2 The offload sweep: quantifying the estimator's error . . . 13

8.3 Generality across models and quantizations . . . 14
8.4 Context length changes the answer . . . 15
8.5 Multi-agent contention and the scheduler ablation . . . 16
8.6 Where the placement metric misleads . . . 16
8.7 End-to-end, through unmodified third-party agents . . . 17
8.8 A prior negative result worth carrying . . . 17

9. Comparison: where our method is better, and where it is not 18

9.1 Positioning against the closest systems . . . 18
9.2 Where our method is better, and on what evidence . . . 19
9.3 Where our method is not better, stated plainly . . . 19

10. Threats to validity 20

11. Open problems 20

12. Conclusion 20

References 22

Appendix A. Reproducing the measurements 24

1. Introduction

Local inference — running a language model on hardware the user owns, with no request leaving the machine — has moved in three years from a curiosity to a default deployment mode for privacy-sensitive, offline, latency-sensitive and cost-sensitive work. The enabling shift was not a single breakthrough but the compounding of several: aggressive post-training quantization that puts a 14-billion-parameter model into nine gigabytes of disk [18, 19, 20]; highly optimized CPU/GPU kernels in llama.cpp and its descendants; and packaging layers such as Ollama that reduce deployment to a single command.

What has *not* kept pace is the tuning of these systems for the hardware they actually run on. A datacenter serving system is tuned continuously against its own workload by people whose job that is. A local installation is tuned once, by a heuristic, at install time, against a hardware profile it infers rather than measures. This survey is about the consequences of that asymmetry, and about which of the literature's many optimization levers are available to a user with one consumer GPU and one interactive agent.

1.1 Scope and the regime under study

We restrict attention to what we will call the *constrained single-machine regime*, defined by four properties that jointly invalidate many datacenter assumptions:

- **Concurrency is one, or nearly one.** A single user drives one or a few agents. Techniques whose benefit comes from amortizing weight reads across a large batch [1, 11, 15] have little to amortize over.
- **Latency, not throughput, is the objective.** The user is waiting. Aggregate tokens per second across requests is not the metric; time to finish *this* task is.
- **Device memory is the binding constraint, and it is small.** An 8 GB card cannot hold a useful 14B model plus its KV cache. Some part of the model must live in host RAM, and the split is the central design decision.
- **The hardware is heterogeneous and unknown in advance.** There is no fleet of identical datacenter cards. There is one machine, with a particular card, a particular driver, and a particular amount of VRAM already consumed by a desktop compositor.

We exclude from scope: training and fine-tuning; multi-GPU and cluster serving except where it provides the contrast that defines our regime; Apple Silicon and other unified-memory architectures, where the GPU/host split this survey is centrally about does not exist in the same form; and model-quality questions such as which quantization preserves reasoning ability, which we touch only where quality interacts with placement.

1.2 Contributions

1. A six-family taxonomy of local-LLM optimization methods (Section 3), with each family assessed explicitly for transfer to the constrained single-machine regime rather than for its reported datacenter gain.
2. A review of each family with representative systems, mechanisms and reported results (Sections 4 and 5).
3. An identification of five gaps the families leave open in this regime, of which the central one is that placement is universally decided by static estimation rather than measurement (Section 6).
4. A description of Ramer, our method, as a runtime layer addressing those gaps, including the algorithm and the two design rules that took real failures to learn (Section 7).
5. An empirical evaluation on one real machine covering placement sweeps, context-length coupling, multi-agent scheduling and end-to-end agentic tasks verified independently of the model's self-report (Section 8).
6. A direct, two-sided comparison against the closest prior systems, stating both where our method is better and where it is redundant or worse (Section 9), followed by threats to validity (Section 10).

1.3 Method of the survey

Literature was gathered by targeted search across the optimization families in September 2026, supplemented by the engineering artefacts that actually govern local deployments — engine issue trackers, design discussions and vendor documentation — which frequently lead the academic literature on this particular topic by a year or more. Where a claim originates in such a source we say so, because its evidentiary weight differs from a peer-reviewed result.

The empirical sections report only measurements taken on the machine described in Section 8.1. Where a result is a single run, we say so. Where a first interpretation was later corrected, we report the correction. Negative results are reported at the same prominence as positive ones; three of the results in Section 8 are negative for our own method.

2. Background: where the time and the memory go

Autoregressive generation has two phases with opposite computational characters, and almost every optimization in the literature is best understood as an attack on one of them.

Prefill processes the entire prompt in one forward pass. Every token is available at once, so the arithmetic is dense matrix-matrix multiplication and the phase is compute-bound. Its cost grows with prompt length, and in a multi-turn agent that re-sends a growing conversation, it is paid again on every turn unless the KV cache is reused.

Decode produces one token at a time. Each step must read the entire set of weights the model uses for that step in order to produce a single token, so the arithmetic intensity is very low and the phase is bound by memory bandwidth, not by floating-point throughput. This single fact explains most of what follows: if decode speed is set by how fast weights can be read, then where the weights live determines the ceiling.

The memory tiers available on a desktop differ in bandwidth by roughly two orders of magnitude end to end. Device memory on a modern consumer card delivers hundreds of gigabytes per second; host DRAM delivers tens; the PCIe link between them delivers tens at best and is shared; NVMe delivers single-digit gigabytes per second; a gigabit network delivers a fraction of one. A weight that must cross from a slower tier on every decode step imposes that tier's bandwidth as the model's effective speed limit. Surveys of resource-efficient LLM systems [38, 39] and of edge deployment [36, 37] both organize their material around this hierarchy, and note the same bandwidth gap of 30 to 50 times between mobile-class and datacenter-class memory systems [36].

Two consequences matter for the rest of this survey. First, *a technique that reduces bytes read per token is worth more in this regime than one that reduces arithmetic*, which is why quantization dominates. Second, *partial offload is not a graceful degradation*: if even a few layers per step must be fetched across PCIe, those fetches serialize against compute and set the pace.

> **The measurement that motivates this survey.** On the test machine, moving a 14B model's layer split from the engine's chosen point to the empirically best point changed decode throughput from 12.21 to 19.64 tok/s, with no change to the model, the quantization, the prompt or the hardware. Moving it eight layers further — past the physical VRAM boundary — dropped it to 4.40 tok/s, below the 5.11 tok/s obtained by using no GPU at all.

3. A taxonomy of optimization methods

We group the literature by *what resource the method changes*, rather than by venue or by era, because that grouping predicts composability: methods in different families generally stack, while methods within a family generally compete.

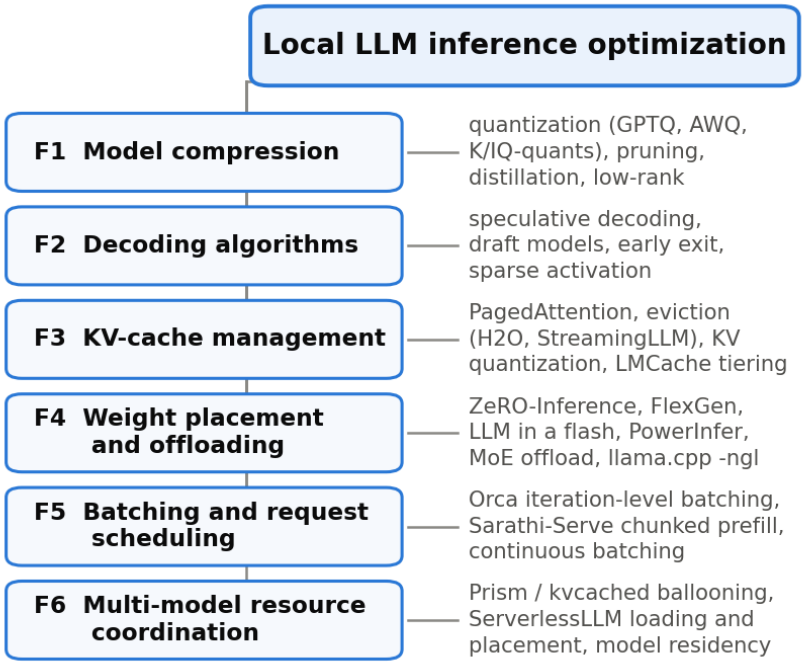


Figure 1. Six families of local-LLM inference optimization. Families compose: a deployment typically applies F1 and F4 together, and may add F3 and F5. The cross-cutting auto-tuning work searches the configuration space the other families expose rather than adding a lever of its own, and is where this survey's own method sits.

Table 1 states each family's mechanism, its representative systems, what it costs, and — the column that matters here — how well it transfers to a single 8 GB consumer GPU serving one interactive agent.

Family	Mechanism	Representatives	Principal cost	Transfer to the constrained regime
F1 Model compression	Fewer bits or fewer parameters per weight, cutting bytes read per token	GPTQ [19], AWQ [20], SmoothQuant [21], K-quants and IQ-quants in llama.cpp, pruning and distillation [38]	Accuracy loss, growing sharply below about 3 bits; some formats cost extra dequantization time	**Essential and universal.** Nothing else brings a 14B model onto an 8 GB card at all. Already applied before any other lever is available.
F2 Decoding algorithms	Produce more than one accepted token per expensive forward pass	Speculative decoding [24, 25], early exit, sparse activation [3, 42]	A second model in memory; gains collapse when the draft model's distribution diverges from the target's	**Partial.** The draft model competes for the very VRAM that is scarce. Attractive in principle, awkward in exactly our regime.
F3 KV-cache management	Reduce, page, evict, quantize or tier the attention state	PagedAttention / vLLM [2], H2O and StreamingLLM [22], KIVI [22, 23], LMCache [16, 17]	Implementation complexity; eviction and compression trade quality for memory	**Partial and indirect.** Paging's headline gain is cross-request sharing, which one user does not have. But the KV cache still competes with weights for the same VRAM, which is central here.
F4 Weight placement and offloading	Decide which tier each weight lives in, and when it moves	ZeRO-Inference [14, 15], FlexGen [1], LLM in a flash [4], PowerInfer [3], Fiddler [5], MoE-Gen [6], MoBiLE [7], llama.cpp layer offload	Transfers on the critical path; throughput-oriented designs trade latency away	**Decisive.** This is the family that determines whether the model runs usably at all, and the one this survey argues is under-tuned in practice.

Family	Mechanism	Representatives	Principal cost	Transfer to the constrained regime
F5 Batching and request scheduling	Order and group work to keep the device busy and avoid stalls	Orca iteration-level batching [11], Sarathi and Sarathi-Serve chunked prefill [12, 13], continuous batching in vLLM [2]	Benefit scales with concurrency; adds scheduling machinery	**Weak.** The mechanism is amortization across concurrent requests, and there is essentially no concurrency to amortize over.
F6 Multi-model coordination	Share one device's memory across several models with residency policy	Prism and kvcached [8, 9], ServerlessLLM [10], CrossPool [50]	Designed for clusters and operators, not for one desktop	**Newly relevant.** Multi-agent desktop workflows recreate the multi-tenancy problem at a scale of one machine, which these systems do not target.

Table 1. The six families, assessed for transfer to a single 8 GB consumer GPU driving one interactive agent rather than for their reported datacenter gains.

4. Review of the families

4.1 F1: model compression

Post-training quantization is the foundation on which local inference rests, and it is the only family whose adoption is effectively universal. Because decode is memory-bound, compressing weights alone reduces memory traffic and therefore accelerates inference, and weight-only schemes require no specialized low-bit operators or hardware support, which is why they dominate practical deployment [18]. GPTQ performs layer-wise quantization by minimizing output perturbation with an approximate second-order method [19]; AWQ protects a small set of salient weight channels selected using activation statistics [20]; SmoothQuant migrates activation outliers into the weights so that both can be quantized [21]. Recent surveys report AWQ and GPTQ as the strongest weight-only results across datasets, with vector quantizers reaching 2-bit regimes at higher integration cost [18].

In the local ecosystem the relevant artefacts are the GGUF K-quant and IQ-quant families rather than the research formats. These matter to this survey for a reason beyond size: *the quantization format changes the compute profile, not only the footprint*. Section 8.6 reports a case where an IQ2_XXS quantization of a 27B model achieved 29.6 tok/s of decode throughput, five times that of a 14B model in a K-quant, and was nonetheless twice as slow on a real task because its prefill cost dominated. Recent work on budget-aware compression pipelines for single-GPU inference makes the same general point, that compression choices couple with each other and with the deployment budget rather than composing cleanly [41].

What this family does not solve. Quantization changes how much must be placed; it does not decide where to place it. A quantized model that still exceeds device memory faces exactly the F4 problem, with the added complication that the quantization has changed the per-layer byte cost the placement heuristic is estimating.

4.2 F2: decoding-algorithm acceleration

Speculative decoding breaks the one-forward-pass-per-token constraint. A small draft model proposes the next several tokens; the target model verifies them in a single parallel pass and accepts the longest agreeing prefix, so the output distribution is provably unchanged while several tokens may be produced per expensive pass [24]. Reported speedups cluster around 1.5 to 2.5 times, and depend jointly on the draft model's own speed and on its acceptance rate, which collapses when the draft's distribution diverges from the target's [24].

Two variants are directly relevant to constrained machines. Dovetail runs the draft model on the GPU and the target on the CPU in a heterogeneous arrangement aimed precisely at consumer hardware [25], and speculative execution has been proposed as a way to hide the offloading latency of mixture-of-experts models behind verification work. Sparse-activation methods are a close cousin: PowerInfer exploits a power-law distribution in neuron activation, preloading consistently active *hot* neurons onto the GPU while computing

input-dependent *cold* neurons on the CPU, reporting 8.32 tok/s on an RTX 4090 across OPT-30B/66B, Falcon-40B and Llama-70B, and 5.77 tok/s on an RTX 2080Ti [3]. CoreInfer pursues the same locality from a semantic angle [42].

The tension in our regime. Every one of these methods spends memory to save time. A draft model resident alongside the target consumes the VRAM that the target's own layers would otherwise occupy, and Section 8 shows that the marginal layer is worth a great deal. This family is not ruled out, but it is not free, and its interaction with placement has not to our knowledge been measured on 8 GB hardware.

4.3 F3: KV-cache management

PagedAttention is the reference point. Kwon and colleagues observed that the KV cache had been allocated as one contiguous region per request, wasting memory to internal and external fragmentation, and applied operating-system paging to it: the cache is stored in non-contiguous blocks, waste falls by an order of magnitude, and blocks may be shared within and across requests. vLLM reported 2 to 4 times the throughput of the previous state of the art at the same latency, and the paper took the SOSP 2023 best-paper award [2].

Three further lines reduce the cache rather than repack it. Token eviction keeps a subset of positions: StreamingLLM retains initial attention-sink tokens plus a sliding recent window, while H2O retains recent tokens plus *heavy hitters* identified by accumulated attention score [22]. Quantization compresses the cache in place; KIVI is a tuning-free asymmetric 2-bit scheme, quantizing keys per channel and values per token, reporting 2.6 times less peak memory and 2.35 to 3.47 times throughput [22]. Surveys note that 8-bit KV is essentially free at under 0.1% perplexity impact, 4-bit is tolerable on keys and costly on values, and recommend eviction and compression as the plug-and-play options requiring no model modification [22, 23]. Finally, tiering moves the cache off the device entirely: LMCache stores KV state across CPU memory, local disk and remote backends so that prefixes can be reused across requests, sessions and engine instances, reporting up to 7 times faster time-to-first-token [16, 17].

Relevance here. The headline benefits are sharing and reuse across many requests, which a single user does not generate in quantity. But one aspect transfers completely and is central to this survey: *the KV cache and the model weights compete for the same device memory*. Doubling the context window on the test machine grew resident VRAM by 0.62 GB, which was enough to push a previously good placement past the physical ceiling and cost a factor of six in throughput (Section 8.4). Any system that decides placement without knowing the context window is deciding it on incomplete information.

4.4 F4: weight placement and offloading

This is the family our method belongs to, and the one whose practical state most diverges from its research state.

Throughput-oriented offloading

ZeRO-Inference pins weights in DRAM or NVMe and streams each layer into device memory as required, deliberately choosing full offload over partial residency because at large batch sizes the resulting batch capacity outweighs the transfer cost [14, 15]. FlexGen formalizes the choice: it aggregates GPU, CPU and disk memory and solves a linear program over storage and access patterns, running OPT-175B on a single 16 GB GPU and reporting up to 100 times the maximum throughput of prior offloading systems [1]. Both are explicit that the objective is throughput for latency-insensitive batch work. That is the opposite of our regime, and the inversion matters: a policy that is optimal for batch 256 can be the worst available policy for batch 1.

Locality-exploiting offloading

A second line asks not how to stream weights faster but how to need fewer of them. Apple's *LLM in a flash* keeps parameters in flash and brings them to DRAM on demand, using windowing to reuse recently activated neurons and row-column bundling to match flash's preference for large contiguous reads, running models

twice the size of available DRAM with 4 to 5 times (CPU) and 20 to 25 times (GPU) the speed of naive loading [4]. PowerInfer, discussed above, applies the same locality insight to the GPU/CPU split [3]. NDP-DIMM proposes attacking the problem in hardware by putting compute near memory [48].

Mixture-of-experts offloading

MoE models present a finer lever, because only a fraction of experts is active for any token. Fiddler offloads expert computation to the CPU rather than moving expert weights to the GPU, reporting up to 10.1 times improvement over prior approaches and 1.26 times in single-batch inference [5]. MoE-Infinity traces expert activation at sequence level and prefetches on the resulting prediction [43]. MoE-Gen batches at module granularity for single-GPU throughput [6], and MoBiLE pairs big and little experts for consumer GPUs [7].

This lever is real, it is larger than the dense-model lever, and — as we report in Section 9.3 — it is *structurally unreachable* from the API our own method is built on, which is a limitation of our method rather than a gap in the literature.

The production heuristic

Against all of this sophistication, what actually ships on consumer machines is a single integer. llama.cpp exposes the number of transformer layers to place on the GPU; Ollama sets it automatically. The automation is the weak point, and the clearest statement of this comes from inside the project. In a December 2025 design discussion, llama.cpp maintainer Johannes Gaessler writes that downstream projects such as Ollama and KoboldCpp have implemented mechanisms for automating memory allocation but that those *rely on rough heuristics and tend to be inaccurate*, and that consequently, to avoid running out of memory in some cases, *the heuristics are rather conservative and leave potential performance on the table* [26]. Practitioner documentation describes the same behaviour concretely, noting that auto-tuning typically reserves about a gigabyte of headroom so that an unrelated application claiming VRAM cannot cause a mid-session spill [26], and that conservative estimation avoids out-of-memory failures while over-reserving memory.

> This survey's central empirical claim is a quantification of that statement. On one machine and three models, the throughput left on the table by conservative estimation was 61%, 6.7% and 402%. The variance across models is itself the finding: an estimator's error is not a constant that could be corrected by a better constant.

4.5 F5: batching and request scheduling

Orca introduced iteration-level scheduling, letting batch composition change between decoding steps so that a finished request leaves immediately and a queued one enters, rather than the batch being locked for the lifetime of its slowest member [11]. Continuous batching in this form is now standard [2]. Sarathi identified the complementary problem, that admitting a long prefill into a batch of running decodes forces the iteration time to scale with the new prompt and stalls every concurrent generation; chunked prefill splits the prompt and interleaves the pieces with decode work, and Sarathi-Serve develops this into a stall-free scheduler that prioritizes ongoing decodes and fills the remaining token budget with prefill [12, 13].

Relevance here. Low, with one exception. Our own earlier tiering experiments confirmed the mechanism directly: increasing batch size improved aggregate multi-tenant throughput without improving any individual user's latency, because batching amortizes weight reads rather than hiding them [44]. For a single interactive agent there is nothing to amortize. The exception is that an agent *does* generate a prefill-heavy request pattern, one new prefill per tool call over a growing conversation, which makes prefill cost rather than decode throughput the thing to optimize — a point we return to in Sections 8.6 and 11.

4.6 F6: multi-model resource coordination

The most direct prior art for part of our method is Prism, which identifies the lack of cross-model memory coordination as the obstacle to sharing a GPU among several models under dynamic load. Its balloon driver,

kvcached, maps token blocks onto virtual and physical GPU pages so that memory can be shared flexibly between models, and a memory-centric control plane adds load-aware placement and slack-aware request arbitration. Prism reports 3.3 times higher throughput and twice the service-level-objective attainment of prior approaches; kvcached is open source and deployed across more than ten thousand GPUs in production [8, 9]. ServerlessLLM attacks the adjacent cold-start problem with a loading-optimized checkpoint format, multi-tier loading, live migration of inference and startup-time-aware scheduling, reporting 6 to 10 times faster loading and 10 to 200 times lower latency than prior serverless systems [10].

Relevance here. The mechanisms are exactly right and the target is exactly wrong. Both systems address operators with clusters; their documentation and evaluation are framed around 80 GB datacenter cards and organizational utilization. Neither targets one developer running four agents against two models on one 8 GB card. That said, the single-machine case is not unserved either: Ollama ships a scheduler that caps concurrently loaded models and evicts on a keep-alive timer [51], and llama-swap is a proxy performing per-request model swapping with TTL unloading and optional co-residency [52]. We state plainly that neither the cluster mechanism nor the desktop mechanism is an open gap; what is open is the residency *policy*, which Section 8.5 measures.

4.7 Cross-cutting: configuration auto-tuning

A recent and fast-growing line of work does not add a lever but searches the space the other families expose. Bench360 evaluates configurations across models, engines, quantization schemes and deployment scenarios specifically to give local deployers actionable guidance [30]. The BentoML llm-optimizer benchmarks and predicts latency, throughput and concurrency limits to identify configurations without exhaustive runs [31], and auto-tuning toolkits search deployment configurations for the greatest completed throughput under an in-flight request cap [32]. Agentic harnesses now rank and tune across vLLM, SGLang, TensorRT-LLM and others [49]. Most significantly for us, llama.cpp itself gained an automated parameter-fitting facility in December 2025: virtual test allocations iteratively reduce memory use until the model fits, feedback loops adjust context size, GPU layer count and tensor split, MoE tensors are deprioritized relative to dense ones, and the reported result is 85 to 90% VRAM utilization across a range of configurations, exposed as new flags and a dedicated tool [26].

A parallel line tries to remove the search rather than automate it, predicting throughput from model metadata so that candidate runs are unnecessary. Qiu et al. fit roofline-shaped predictors over GGUF metadata and report held-out decode error of 13.1% and 14.4% on two unified-memory Apple systems but 36.1% on a discrete NVIDIA GPU, with a prefill baseline reaching 108.2% there, concluding that fitted efficiencies are not universal [55]. That result is worth noting carefully: the estimator degrades precisely on the hardware class this survey is about, which is independent support for measuring rather than predicting in this regime.

This is the family our method belongs to, and reference [26] is the closest thing in existence to a competitor. Section 9.2 compares them directly rather than avoiding the comparison.

5. How the families compose, and what a local stack actually uses

Families compose along the resource they touch. A typical well-configured local deployment in 2026 applies F1 (a 4-bit K-quant), F4 (a layer split between GPU and CPU), and implicitly F3 (whatever cache management the engine performs), while leaving F2, F5 and F6 unused. The unused families are not unused by oversight: F2 costs memory that is scarce, F5's benefit requires concurrency that is absent, and F6's implementations target operators.

The important structural observation is that *F1 and F4 interact*. Quantization sets the per-layer byte cost, which is the input to every placement heuristic, and different quantization families have different compute profiles at the same size. A heuristic calibrated on K-quants is not obviously calibrated on IQ-quants. Our own measurements bear this out: the estimation error we measured was 61% on a 14B K-quant, 6.7% on a 32B

K-quant at the VRAM boundary, and 402% on a 27B IQ-quant (Section 8.3). An error that varies by two orders of magnitude across formats is the signature of a model-based estimator meeting a case outside its calibration.

6. Five gaps in the constrained single-machine regime

6.1 Placement is estimated, never measured

Every production engine computes a placement from a model of memory consumption: file size, layer count, an estimate of KV cache size, a safety margin. No widely-deployed local engine, before [26], validated that estimate against the machine by running the model and timing it, though practitioners sweep the parameter by hand with the engine's own benchmarking tools [53], and MCAP profiles on the target device at load time to drive per-layer residency, using a hardware-independent importance signal rather than a throughput measurement [54]. The estimate must be conservative because its failure mode is an out-of-memory crash, and conservatism has a cost that is invisible precisely because nothing measures it [26]. The user sees a working system and has no way to know it is 40% slower than it could be.

6.2 Placement validity is coupled to context length, and this is ignored

Weights and KV cache share device memory, so the largest weight placement that fits is a function of the context window. A placement measured or estimated at one context is not merely suboptimal at another; it can be *wrong*, in the sense of pushing total residency past the physical ceiling. We are not aware of any local engine or tuning tool that treats the context window as part of the placement's identity, though [26] does fit context size jointly.

6.3 The oversubscription cliff is undefended

On Windows, the display driver model manages device memory as a demand-paged virtual memory system, and since driver 536.40 an allocation that exceeds device memory silently falls back to system memory rather than failing [28, 29]. The application keeps running, at dramatically lower speed; NVIDIA's own documentation of oversubscription performance notes that scattered access patterns in the oversubscription domain can degrade to a few hundred kilobytes per second of effective read bandwidth because of page faulting and migration [29]. For an inference engine this converts a hard, diagnosable failure into a soft, silent one. Our measurements show the consequence: requesting all 48 layers of a 14B model on an 8.15 GB card, needing 9.30 GB, produced 6.17 tok/s on one run and 4.40 tok/s on a second, against 19.64 tok/s at the best in-VRAM placement and 5.11 tok/s using no GPU at all.

The non-determinism is the hard part. Section 8.4 reports a case where a placement reporting 8.52 GB on the same 8.15 GB card did *not* collapse, sustaining 13.93 tok/s. Crossing the ceiling is necessary but not sufficient for the cliff, which means no static estimate — ours included — can predict it reliably, and a runtime defence is required.

6.4 Single-machine multi-agent contention is managed, but not by policy

Desktop AI workflows have become plural: a coding agent, an assistant, a background job. Each independently asks the local engine for a model, and the engine, having no notion of which caller matters, evicts and reloads to serve whoever asked most recently. Residency itself is not unmanaged: Ollama's scheduler caps concurrently loaded models and unloads on a keep-alive timer [51], and llama-swap proxies per-request model swapping with time-to-live unloading [52]. What neither provides is priority between callers or affinity-aware admission, which is where the cost in Section 8.5 comes from.

6.5 Agentic workloads are not benchmark workloads

Optimization work is overwhelmingly evaluated on synthetic generation with fixed prompt and output lengths. Agent workloads behave differently, and a 2026 literature has begun to characterize how. One characterization finds that non-LLM components dominate latency in five of ten studied applications, that agentic execution is heavyweight and stateful, and that components have heterogeneous resource affinity, some GPU-bound, some memory-bound, some CPU-bound [34]. Another finds that ReAct-style execution is dominated by repeated model invocations but that tool latency remains a substantial, workload-dependent component, and that with effective context caching most input tokens are reused across turns, making execution decode-dominated while increasing dependence on long-lived KV state [33]. A third characterizes coding-agent traces specifically for serving-system design [35].

The implication for any inference-side optimization is uncomfortable and we adopt it as a constraint rather than working around it: *if a substantial share of an agent's wall time is not local inference, no inference optimization can improve that share*. Section 8.7 measures this for our own method and reports the negative result it produces.

7. Our method: Ramer

Ramer is a runtime layer that sits between an agent tool and a local inference engine. It addresses gaps 6.1 through 6.4 and is constrained by 6.5. It adds no new offloading mechanism, no kernel, no quantization format; it decides, per model and per context length, how to configure the mechanism the engine already has, and arbitrates access when several agents want it at once.

7.1 Design principles

1. **Measure, do not estimate.** The placement is chosen by running the model at candidate splits and timing decode, once per model and context, then caching the winner. The cost is a one-time probe; the benefit is an answer that is correct for this machine including whatever else is resident on it.
2. **A placement is only valid at the context it was measured at.** The context window is part of the cache key, not a parameter of the workload. This rule was adopted after a real failure, not by foresight; see 7.3.
3. **Prefer headroom when throughput ties.** Among candidates within the measurement noise floor of the fastest, take the one with the smallest VRAM footprint. Also adopted after a real failure; see 7.3.
4. **Defend at run time, because the cliff is not predictable.** A cheap pre-flight check compares live throughput against the cached figure and falls back to the engine's own safe placement if it has regressed, flagging the cache for re-probing.
5. **Be invisible.** Integration is a base-URL change in an existing tool. A layer that requires users to adopt a new agent is a layer most users will not adopt.

7.2 The placement algorithm

For a model m and context window c, Ramer sweeps candidate GPU layer counts, and for each runs a bounded generation, recording decode tokens per second and the engine's reported resident VRAM. Selection is not an argmax over throughput. Writing T for the best observed throughput and δ for the noise tolerance, the chosen candidate is the one of smallest VRAM footprint among those whose throughput is at least $T(1 - \delta)$. The result is cached under the pair (m, c), so a split measured under one memory pressure can never be silently reused under another. Cache entries from a different context simply fail to match and are re-probed.

At run time, before a session, a short generation is timed against the cached figure. A sustained shortfall triggers a fallback to the engine's own automatic placement for that run and marks the entry stale.

7.3 Two rules that were learned, not designed

We report these as failures because they are the evidence for the rules.

The context-key rule. A placement probed at a 4096-token context was reused at 8192 tokens by an agent that needed to read real source files. The first model call returned 8.71 tok/s and the second 1.47 tok/s. The cause was measured, not inferred: the same split occupied 7.27 GB at 4096 and 7.89 GB at 8192, the difference being the doubled KV cache, and the larger figure sits past the card's usable ceiling. Re-probing at the correct context took the same task from 85.9 s to 24.1 s, and the call that had collapsed from 1.47 to 12.09 tok/s.

The headroom rule. On the first context-aware probe, the raw argmax selected 41 layers at a reported 8.68 GB — already past the physical ceiling and being paged — over 33 layers at 7.12 GB, because it measured 0.04 tok/s faster. Noise had purchased a zero-headroom placement, which is the precise configuration every collapse in this project has come from. The tie-break rule was verified offline against the same grid, then by a forced re-probe.

7.4 Multi-agent scheduling

Concurrent agents are serialized through a single chokepoint that tracks which model is resident, orders waiting requests by caller priority, and applies *sticky affinity*: a request for the already-resident model is preferred, and a brief grace window is held before paying a model switch. This was built after reproducing the contention failure directly, and its two mechanisms were separated by ablation rather than assumed complementary (Section 8.5).

7.5 Integration

Ramer exposes a proxy speaking both the local engine's native protocol and the OpenAI-compatible chat API, including the tool-calling paths on both, which differ in whether call arguments arrive as parsed objects or as JSON strings. An existing agent tool changes one base URL. The evaluation in Section 8.7 uses two third-party tools unmodified, which is the only integration claim that we consider meaningful.

8. Empirical evaluation

8.1 Setup and honesty protocol

All measurements were taken on one machine: an NVIDIA RTX 5060 with 8151 MiB of VRAM (Blackwell, sm_120), a Ryzen 5 7600 with 6 cores and 12 threads, 32 GB of system RAM, running Windows 11. The inference engine is Ollama over llama.cpp. Models are Qwen2.5-14B-Instruct Q4_K_M, Qwen2.5-32B Q2_K, and Qwen3.8-27B in an unsloth UD-IQ2_XXS quantization, plus smaller Qwen2.5 models for the scheduling experiments.

Four protocol rules were applied throughout, and are stated so that the results can be discounted appropriately. Measurements are never simulated or estimated; a number that could not be measured is absent rather than modelled. Agentic task outcomes are verified externally, by inspecting the version-control diff and running the test suite, never by trusting the agent's or the model's own report — a rule adopted after a session that wrote zero files reported having modified three. Single runs are labelled as such. Negative results are reported.

8.2 The offload sweep: quantifying the estimator's error

We swept the GPU layer count for the 14B model at a 4096-token context, with one clean model reload per point, and compared against the engine's own automatic choice.

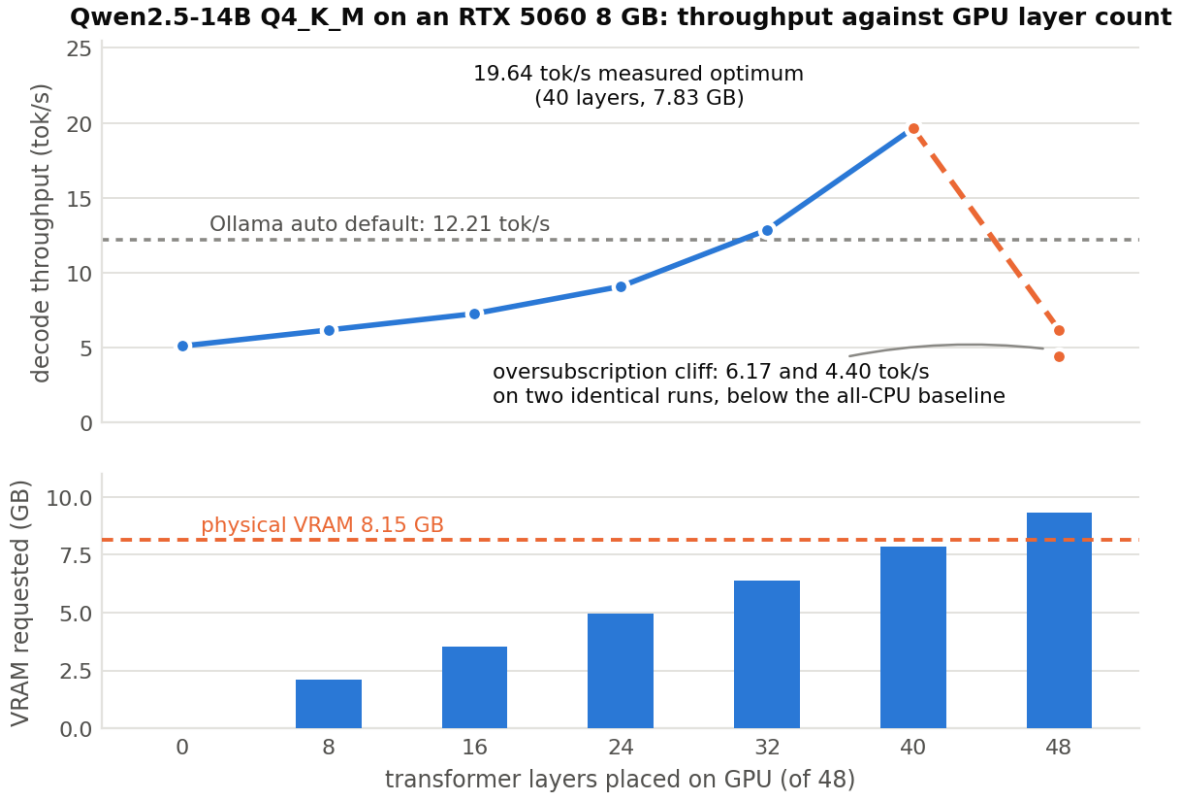


Figure 2. Decode throughput (top) and requested VRAM (bottom) against GPU layer count for Qwen2.5-14B Q4_K_M. Within the engine's supported range the curve is smooth and monotonic, which is the engine working correctly. The dashed line marks the engine's automatic choice at 12.21 tok/s; the measured optimum at 40 layers reaches 19.64 tok/s and still fits in physical VRAM. Requesting all 48 layers needs 9.30 GB on an 8.15 GB card and collapses.

Three findings. First, *the engine's offloading is good*: from 0 to 40 layers the curve is clean and monotonic in both decode and prefill, with no software cliff. This is worth stating because it refutes the premise this project started from, that offloading was fundamentally broken. Second, *the automatic placement is conservative by 61%*: it chose a 6.20 GB split for 12.21 tok/s when a 7.83 GB split delivering 19.64 tok/s was available entirely within physical VRAM. Third, *the boundary is a cliff, not a slope*: the 48-layer request produced 6.17 and 4.40 tok/s on two identical runs, and live sampling during the second confirmed the mechanism, with GPU memory pinned at 7758 MiB and utilization at 95 to 100% throughout, consistent with the driver paging the overflow through system memory [28, 29]. The run-to-run variance at an identical request is itself evidence of paging rather than deterministic slowdown.

8.3 Generality across models and quantizations

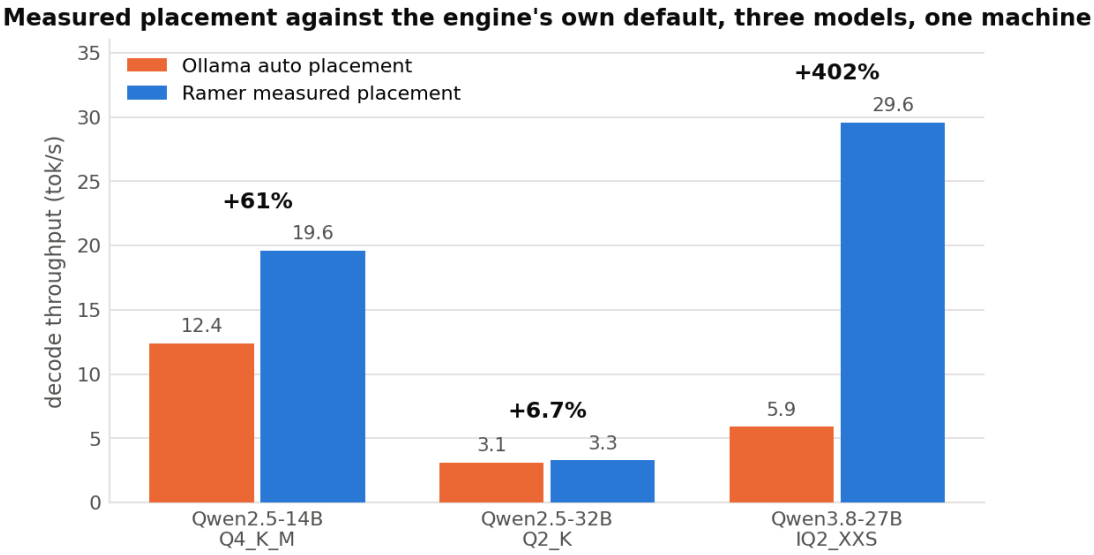


Figure 3. Measured placement against the engine's default on three models. Values are the summary figures recorded in the repository's results table; Section 8.2 gives the full sweep behind the 14B row. The spread from 6.7% to 402% is the finding: estimator error is not a constant.

The 27B IQ2_XXS case is the extreme. The engine's automatic placement left about 2.3 GB of the card's own memory unused rather than fully offloading a model that, once measured, fit: 5.903 tok/s against 29.607 tok/s at full offload, a gain of 402%. The 32B Q2_K case is the opposite extreme, at 6.7%, because that model is 12.3 GB and never achieves full offload on this card; when there is no better placement available, measurement correctly reports that there is none.

We draw the conclusion narrowly. *The size of the opportunity is unpredictable without measuring, which is itself the argument for measuring.* A tool that promises a fixed improvement would be wrong in both directions here.

8.4 Context length changes the answer

Holding the model and hardware fixed, we swept only the context window, running a full fresh probe at each point and comparing with the engine's automatic choice.

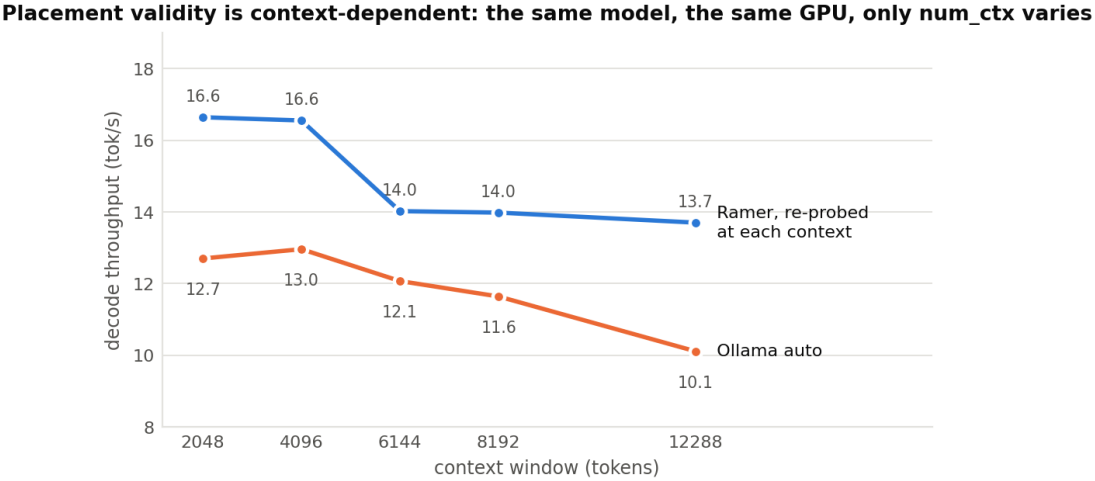


Figure 4. Context-length sweep for Qwen2.5-14B Q4_K_M. The automatic placement degrades from 12.70 to 10.11 tok/s as context grows, apparently holding back layers to leave room for the KV cache, while re-probing at each context holds a 13.7 to 16.6 tok/s band. The best layer count itself moves, from 37 at 2048 tokens to 33 at 6144 and 8192.

The gain over the default persists at every context tested, between 16.2% and 35.5%, with no trend toward vanishing as context grows. Inferred KV-cache growth was smooth, 1.54 GB across a six-fold context

increase, which by itself would have been unremarkable.

The negative result in this experiment is the more important one. A fixed 37-layer placement held at 12288 tokens reported 8.521 GB of resident VRAM, above the card's physical capacity, and did not collapse: 13.93 tok/s, close to its small-context baseline. The same model at the same layer count had collapsed to 1.47 tok/s at a smaller context reaching only 7.89 GB. Crossing the ceiling is therefore necessary but not sufficient for the cliff, and what determines which outcome occurs — external GPU load, driver page-out timing, which pages are evicted — is not captured by any static quantity we can probe. This bounds our own claim: context-aware placement removes placements that are flatly wrong, and cannot be sold as preventing the cliff. It is also why the runtime check in 7.2 is load-bearing rather than belt-and-braces.

8.5 Multi-agent contention and the scheduler ablation

Four agents with different models, tasks and workspaces were run concurrently under four conditions, five trials each, to separate what sticky affinity and priority each contribute.

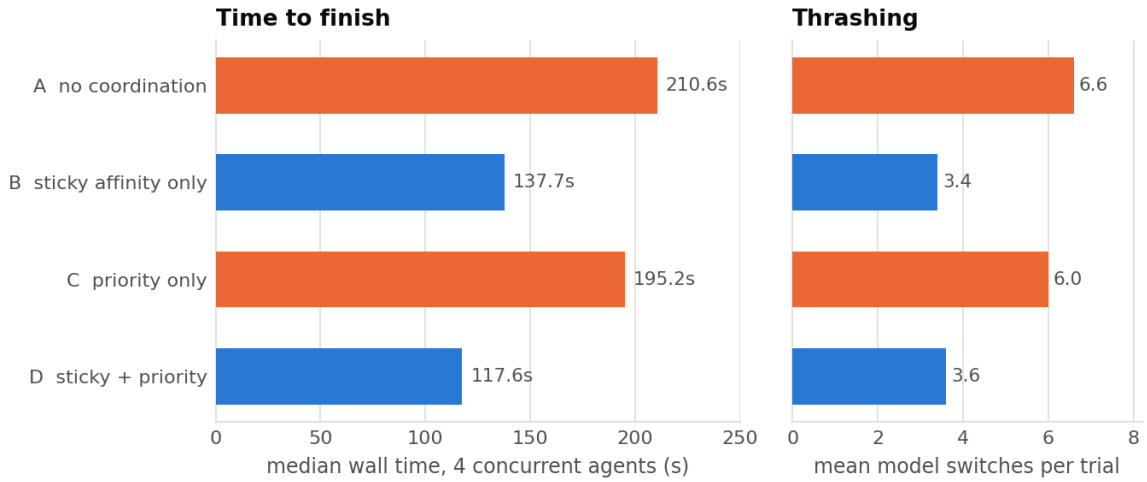


Figure 5. Four-way ablation over five trials per condition. Blue marks the two conditions that include sticky affinity. Left: median wall time for the whole set of agents. Right: mean model switches per trial, the direct measure of thrashing.

Sticky affinity, not priority, reduces thrashing. Affinity alone cuts mean switches from 6.6 to 3.4, essentially matching the full scheduler's 3.6, while priority alone barely moves it, at 6.0. The efficiency gain is therefore a model-residency effect, not a scheduling effect.

Priority protects interactive agents, but only on top of affinity. Affinity alone takes the coding agent from a 201.6 s to a 113.4 s median; adding priority takes it to 38.6 s. Priority *without* affinity is a bad trade: it improves total time by only 7% while taking the lowest-priority agent from 76.3 s to 195.2 s, the worst outcome for any agent in any condition tested. The two mechanisms are complementary and each is weak alone.

Confidence, stated unevenly on purpose. The coordinated-versus-uncoordinated comparison is clean, with five of five non-overlapping trials. The four-way decomposition is directionally consistent across every metric but its trial ranges overlap; the fastest priority-only trial beats four of five full-scheduler trials. Treat the pattern as supported and the exact effect sizes as approximate.

8.6 Where the placement metric misleads

The 27B IQ2_XXS model, at its measured best placement, completed a real bug-fixing task correctly: it diagnosed the fault, read both files, wrote the fix, re-ran the test suite itself, and external verification confirmed two passing tests. It took 313.5 s. The 14B K-quant model completed the identical task in 157.8 s, despite decoding at a fifth of the speed.

The transcript shows why. Several turns logged a single token over 126, 43 and 41 seconds; the first turn alone consumed 126 of the 313.5 seconds. The bottleneck was time-to-first-token, that is prefill, and our probe measures bounded-length decode throughput, which does not observe prefill at all. Whether the cost is inherent to IQ-quant dequantization, specific to this quantization and context pairing, or an artefact of a tight KV budget for a 64-layer model is not established by one run and we do not claim it.

> **This is a gap in our own method, reported because it changes how the numbers in Figure 3 should be read.** A large measured decode gain does not imply a proportionate end-to-end gain, because an agent pays a fresh prefill on every turn over a growing conversation. Prefill-aware probing is the most concrete item of future work in Section 11.

8.7 End-to-end, through unmodified third-party agents

The only test we consider decisive: take a third-party agent CLI, point it at the engine, run a task; point it at Ramer, run the same task; verify the outcome externally both times.

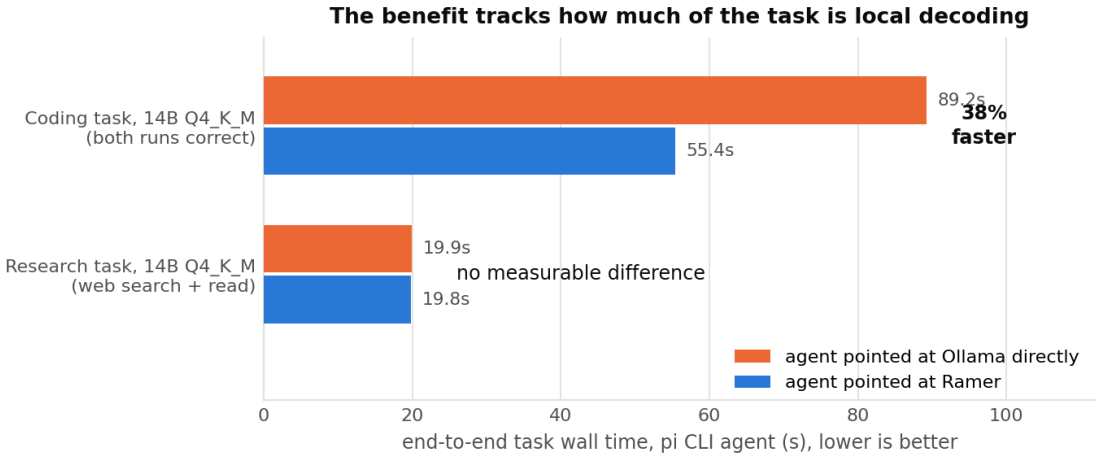


Figure 6. Identical tasks through an unmodified third-party agent, with and without Ramer in the path. Both coding runs produced correct, externally verified output. The research task is a deliberate contrast case where tool and network latency dominate.

On a coding task with the 14B model, 55.4 s against 89.2 s, both correct: a 38% end-to-end improvement. On a research task requiring web search and page reading, 19.8 s against 19.9 s: no benefit, because the wall time is network and tool latency, which no inference-side optimization can compress. On a coding task with the 32B Q2_K model, neither condition succeeded; placement cannot compensate for a model that is inadequate to the task.

These three cases, two of them negative, are what we consider the defensible form of the claim. They are also consistent with the independent agentic characterization literature, which finds tool latency to be a substantial and workload-dependent share of agent execution [33, 34].

8.8 A prior negative result worth carrying

This work grew out of an earlier ten-phase study of remote and tiered memory for local inference [44], whose conclusion is directly relevant to the families surveyed above and is reported here because it is a negative result about a popular idea.

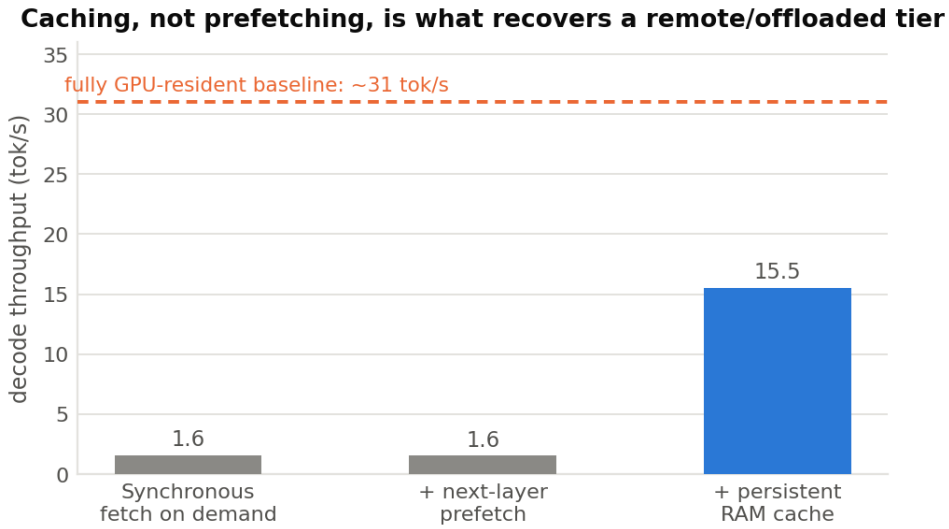


Figure 7. Layer-tiering strategies under a simulated remote tier, with four of 28 layers held off-device. Per-layer prefetch buys nothing because the compute window available to hide a transfer behind is roughly a hundred times too small. Persistent caching of the immutable weights recovers most of the loss, at a 99.9% hit rate.

The generalizable finding is that *caching beat prediction*. Prefetch-ahead at layer granularity failed at every batch size tested; a persistent RAM cache of weights, which are immutable between tokens, took throughput from 1.6 to 15.5 tok/s against a 31 tok/s fully-resident baseline. Under a changing multi-tenant workload, the strongest baseline in the entire study was reactive shared-LRU caching with no prediction at all, which outperformed every predictive policy tested. We report this because the original hypothesis of the project was that learned prefetching would be the mechanism that worked, and it was not.

9. Comparison: where our method is better, and where it is not

9.1 Positioning against the closest systems

System	Decides placement by	Context-aware	Runtime re-validation	Multi-model policy	Target
Ollama automatic placement	Static estimate with a fixed safety margin [26]	Estimated, not keyed	None	Evict on demand	Any local machine
llama.cpp *--fit* / llama-fit-params [26]	Virtual test allocations plus feedback on memory fit	Yes, fits context jointly	None described	Single process	Any llama.cpp deployment
FlexGen [1]	Linear program over a cost model	Fixed in the plan	None	Single model	Batch throughput, latency-insensitive
PowerInfer [3]	Offline activation profiling, hot/cold split	Not the governing factor	None	Single model	Consumer GPU, dense-sparse
Prism / kvcached [8, 9]	Runtime memory ballooning across models	KV-centric by design	Continuous control plane	**Full: placement, arbitration**	Clusters, 10k+ GPUs
ServerlessLLM [10]	Locality-aware model scheduling	Not addressed	Live migration	Full, across servers	Serverless clusters
Ramer (ours)	**Empirical probe of real throughput, cached**	**Yes: context is part of the cache key**	**Pre-flight check, fallback, cache invalidation**	**Sticky affinity plus priority, one machine**	**One consumer GPU, several local agents**

Table 2. Ramer against the systems nearest to it on the axes that distinguish them. Bold marks where our method's position differs from every other row; it is not a claim of superiority on the other axes.

9.2 Where our method is better, and on what evidence

1. **Measurement instead of estimation.** Every other local-deployment path computes placement from a model of memory use. Ramer runs the model and times it. The evidence that this matters is the 6.7% to 402% spread in Figure 3: the estimator's error is not a constant, so no better constant fixes it.
2. **Context as part of the placement's identity.** We are not aware of another local tool that refuses to reuse a placement across context windows. The evidence is the 8.71 to 1.47 tok/s collapse in 7.3, which is a correctness failure rather than a missed optimization.
3. **A headroom-aware tie-break.** Selecting for throughput alone lets measurement noise buy a zero-headroom placement, which we observed directly. Preferring the smaller footprint among statistically indistinguishable candidates is a one-line rule with a documented failure behind it.
4. **A runtime defence for a non-deterministic failure.** Section 8.4 shows that crossing the VRAM ceiling sometimes collapses throughput and sometimes does not. No static configuration can defend against that; a pre-flight throughput check and fallback can.
5. **Multi-agent arbitration at the scale of one machine.** A 210.6 s to 117.6 s median improvement with switches halved, and an ablation separating which mechanism does what. Existing desktop residency managers [51], [52] swap and evict but do not order callers by priority or admit by affinity; the contribution here is the policy and its ablation, not the mechanism.
6. **Validation through unmodified third-party tools with external verification.** The end-to-end results use agents we did not write, on tasks whose success is checked by a test suite and a version-control diff rather than by the model's self-report.

9.3 Where our method is not better, stated plainly

- **An upstream facility now attacks the same problem from inside the engine.** The llama.cpp parameter-fitting work of December 2025 performs virtual test allocations, fits GPU layers, tensor split and context size jointly, and reports 85 to 90% VRAM utilization [26]. It sits below us in the stack, benefits every downstream consumer at once, and handles multi-GPU and MoE tensor overrides that we do not. Our remaining distinctions are that we measure throughput rather than fit memory, that we re-validate at run time, and that we arbitrate across agents — genuine but narrower than the claim we could have made a year earlier.
- **The MoE lever is structurally out of reach.** Keeping shared attention on the GPU while routing individual experts to the CPU is a larger win than the dense-layer split, it is supported by llama.cpp directly, and the API we build on does not expose it. We confirmed this three ways, including a direct sweep on a real MoE model whose VRAM scaled in smooth per-block steps with no expert-awareness. Reaching it means driving a different backend, not tuning ours harder.
- **The multi-tenant mechanism is solved better elsewhere.** Prism's shared budget with eviction and arbitration is more sophisticated than ours and runs in production at a scale we cannot approach [8, 9]. Our contribution on this axis is target and packaging, not mechanism.
- **We contribute nothing to compression, decoding algorithms or KV-cache compression.** Families F1, F2 and most of F3 are orthogonal to our work and we simply inherit whatever the engine and the model file provide.
- **The probe does not measure prefill.** Section 8.6 gives a case where this made a 402% decode gain fail to translate into an end-to-end win.
- **The benefit is workload-dependent and can be zero.** On tool-latency-bound work we measured no difference at all, and we expect none.
- **Evidence is from one machine.** Every number here is from a single RTX 5060 system. The effects are mechanistic and we expect them to generalize in shape, but we have not demonstrated that.

> **The claim we defend.** Not that Ramer makes local AI faster. That *measurement beats estimation for weight placement on memory-constrained single-GPU machines; that the measurement must be keyed to context length; and that the resulting benefit is proportional to the share of a workload that is genuinely local decoding*. Each clause is separately falsifiable, and the third one bounds the first two.

10. Threats to validity

- **Single-machine, single-GPU evidence.** One card, one driver, one operating system. The oversubscription cliff in particular is a Windows display-driver behaviour [28, 29] and its form on Linux is untested.
- **Small sample sizes.** The scheduler ablation is five trials per condition and its per-condition ranges overlap. The agentic comparisons are single runs per condition on a small number of tasks.
- **Measurement instrument limits.** Model-switch counts come from 0.5 Hz polling, consistent across conditions but approximate in absolute terms. Resident-VRAM figures are the engine's own reports, which Section 8.4 shows can exceed physical capacity and therefore describe requested rather than resident memory.
- **A framework artefact that affects absolute numbers elsewhere in the project record.** An earlier phase of this work measured a 25-fold offload penalty under a research framework; measuring the production engine at comparable offload severity on the same machine gave roughly fifteen times the throughput. The relative shapes from that phase stand; its absolute figures should not be quoted outside their framework context. We flag this because the figure has been quoted internally as motivation.
- **Benchmark-to-task gap.** Our own probe optimizes a quantity (bounded decode throughput) that Section 8.6 shows can diverge from the quantity users care about (task completion time).
- **Selection of comparison points.** We compare against the engine's automatic placement, which is the realistic default, not against an expert hand-tuned configuration, which a small number of users do produce.

11. Open problems

1. **Prefill-aware placement.** The clearest gap in our own method. A probe that measures time-to-first-token alongside decode throughput, and a selection rule that weights them by the observed shape of the workload, would have caught the case in Section 8.6.
2. **Predicting the oversubscription cliff.** Nothing static that we measured predicts whether a given overflow degrades smoothly or collapses. Understanding what does would convert a runtime defence into a design-time guarantee.
3. **Placement under agentic workload shape.** The characterization literature [33, 34, 35] describes agent workloads well enough to design for them; no placement policy we are aware of consumes that description.
4. **MoE-aware placement on consumer hardware.** The expert-routing lever is the largest unexploited one in this regime, and the gap between what llama.cpp supports and what packaging layers expose [27] is a practical, not a research, obstacle.
5. **Economic evaluation.** The honest framing of the whole line of work: memory orchestration competes with buying more memory. Establishing where orchestration is genuinely cheaper than capacity is a question this survey does not answer.

12. Conclusion

The literature on optimizing large language model inference is deep, active and largely aimed elsewhere. Quantization transfers to consumer hardware completely and is the reason local inference exists at all. KV-cache paging, continuous batching, chunked prefill and cross-model memory ballooning are excellent work whose mechanisms depend on concurrency, scale or operator control that a single desktop does not supply. Offloading research has concentrated on throughput for batch workloads, while the offloading that consumer machines actually perform is governed by a conservative static estimate whose error nobody measures.

We measured that error on one machine and found it to be between 6.7% and 402% depending on the model, to depend on context length in a way that can make a placement wrong rather than merely suboptimal, and to be bounded by a driver-level paging cliff that is non-deterministic and therefore undefendable at design time. Replacing the estimate with a cached empirical probe, keying it to context, preferring headroom on ties, re-validating at run time, and arbitrating residency across concurrent agents produced measurable end-to-end improvements on decode-heavy work and, equally importantly, measurable non-improvements elsewhere.

The last point is the one we would most like carried forward. An optimization layer for local inference is worth exactly the share of the user's wall time that is local inference. On a coding task that share was large and the improvement was 38%. On a research task dominated by network and tool latency the share was small and the improvement was nil. Reporting both is what makes the first number worth anything.

References

[1] Sheng, Y., Zheng, L., Yuan, B., et al. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. ICML 2023. arXiv:2303.06865.

[2] Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., Stoica, I. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023 (best paper). arXiv:2309.06180.

[3] Song, Y., et al. PowerInfer: Fast Large Language Model Serving with a Consumer-grade GPU. SOSP 2024. arXiv:2312.12456.

[4] Alizadeh, K., et al. LLM in a flash: Efficient Large Language Model Inference with Limited Memory. Apple Machine Learning Research. arXiv:2312.11514.

[5] Kamahori, K., et al. Fiddler: CPU-GPU Orchestration for Fast Inference of Mixture-of-Experts Models. ICLR 2025. arXiv:2402.07033.

[6] MoE-Gen: High-Throughput MoE Inference on a Single GPU with Module-Based Batching. arXiv:2503.09716.

[7] MoBiLE: Efficient Mixture-of-Experts Inference on Consumer GPU with Mixture of Big Little Experts. arXiv:2510.12357.

[8] Yu, S., et al. Prism: Cost-Efficient Multi-LLM Serving via GPU Memory Ballooning. OSDI 2026. arXiv:2505.04021.

[9] kvcached: virtual-memory ballooning for LLM serving. https://github.com/ovg-project/kvcached

[10] Fu, Y., et al. ServerlessLLM: Low-Latency Serverless Inference for Large Language Models. OSDI 2024. arXiv:2401.14351.

[11] Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., Chun, B.-G. Orca: A Distributed Serving System for Transformer-Based Generative Models. OSDI 2022.

[12] Agrawal, A., et al. SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills. arXiv:2308.16369.

[13] Agrawal, A., et al. Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve. OSDI 2024. arXiv:2403.02310.

[14] Aminabadi, R. Y., et al. DeepSpeed Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. arXiv:2207.00032.

[15] DeepSpeed Team. ZeRO-Inference: Democratizing massive model inference. https://www.deepspeed.ai/2022/09/09/zero-inference.html

[16] LMCache: An Efficient KV Cache Layer for Enterprise-Scale LLM Inference. Technical report. https://lmcache.ai/tech_report.pdf

[17] LMCache project repository. https://github.com/LMCache/LMCache

[18] Low-bit Model Quantization for Deep Neural Networks: A Survey. arXiv:2505.05530.

[19] Frantar, E., Ashkboos, S., Hoefler, T., Alistarh, D. GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers. arXiv:2210.17323.

[20] Lin, J., et al. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. arXiv:2306.00978.

[21] Xiao, G., et al. SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models. arXiv:2211.10438.

[22] Rethinking Key-Value Cache Compression Techniques for Large Language Model Serving. arXiv:2503.24000.

[23] KV Cache Optimization Strategies for Scalable and Efficient LLM Inference. arXiv:2603.20397.

[24] Speculative Decoding with CTC-based Draft Model for LLM Inference Acceleration. NeurIPS 2024.

[25] Dovetail: A CPU/GPU Heterogeneous Speculative Decoding for LLM Inference. arXiv:2412.18934.

[26] Gaessler, J. Automation for GPU layers, tensor split, tensor overrides, and context size (with MoE optimizations). llama.cpp discussion #18049, 15 December 2025. https://github.com/ggml-org/llama.cpp/discussions/18049

[27] Ollama issue #11005: dynamic expert loading for MoE models (open, unimplemented). https://github.com/ollama/ollama/issues/11005

[28] NVIDIA. System Memory Fallback for Stable Diffusion. NVIDIA support answer 5490.

[29] NVIDIA. Improving GPU Memory Oversubscription Performance. NVIDIA Technical Blog.

[30] Bench360: Benchmarking Local LLM Inference from 360 degrees. arXiv:2511.16682.

[31] BentoML llm-optimizer: benchmark and optimize LLM inference across frameworks. https://github.com/bentoml/llm-optimizer

[32] llm-tuning-toolkit: automatic tuning and benchmarking of LLM serving configurations. https://github.com/juanjucm/llm-tuning-toolkit

[33] Agentic AI Workload Characteristics. arXiv:2605.26297.

[34] From LLM Inference to Agentic Workloads: Characterization and Implications for Serving Systems. arXiv:2608.15127.

[35] TraceLab: Characterizing Coding Agent Workloads for LLM Serving. arXiv:2606.30560.

[36] A Review on Edge Large Language Models: Design, Execution, and Applications. ACM Computing Surveys. arXiv:2410.11845.

[37] Deploying LLM Transformer on Edge Computing Devices: A Survey of Strategies, Challenges, and Future Directions. MDPI AI, 7(1), 15.

[38] Beyond Efficiency: A Systematic Survey of Resource-Efficient Large Language Models. arXiv:2401.00625.

[39] LLM Inference Serving: Survey of Recent Advances and Opportunities. arXiv:2407.12391.

[40] Memory Offloading for Large Language Model Inference with Latency SLO Guarantees. arXiv:2502.08182.

[41] Budget-Aware Compression Pipeline for Single-GPU LLM Inference: Methods, Trade-offs, and Coupling Effects. arXiv:2608.30076.

[42] CoreInfer: Accelerating Large Language Model Inference with Semantics-Inspired Adaptive Sparse Activation. arXiv:2410.18311.

[43] MoE-Infinity and related expert-offloading systems, indexed in the MoE inference paper collection. https://github.com/MoE-Inf/awesome-moe-inference

[44] Ramer project. Adaptive Memory Tiering for Local LLM Inference: An Experimental Study of Remote Memory, Prefetching, and Persistent Caching. docs/ARCHITECTURE.md, ten phases, 2026.

[45] Ramer project. Product hypotheses, experiments and corrections. PRODUCT_HYPOTHESES.md, 2026.

[46] Ollama. https://ollama.com

[47] ggml-org/llama.cpp. https://github.com/ggml-org/llama.cpp

[48] Make LLM Inference Affordable to Everyone: Augmenting GPU Memory with NDP-DIMM. arXiv:2502.16963.

[49] Auto Inference: agentic LLM inference optimization CLI. https://autoinference.org/

[50] CrossPool: Efficient Multi-LLM Serving for Cold MoE Models through KV-Cache and Weight Disaggregation. arXiv:2606.24506.

[51] Ollama. Request scheduling and runner management: OLLAMA_MAX_LOADED_MODELS and OLLAMA_KEEP_ALIVE. https://ollama.com

[52] llama-swap. Reliable model swapping for any local OpenAI-compatible server. https://github.com/mostlygeek/llama-swap

[53] ggml-org. llama-bench and llama-sweep-bench, llama.cpp. https://github.com/ggml-org/llama.cpp

[54] Das, A. MCAP: Deployment-Time Layer Profiling for Memory-Constrained LLM Inference. arXiv:2604.21026, 2026.

[55] Qiu, X., Xu, C., Su, B., Chen, Z., Xu, R., Dai, S. GGUF-Metadata Prediction of Single-Sequence llama.cpp Throughput Across Three Systems. arXiv:2609.14864, 2026.

Appendix A. Reproducing the measurements

Every figure in this survey is generated from numbers recorded in this repository. The table below maps each result to its source.

Result	Source in this repository
Offload sweep, Figure 2	PRODUCT_HYPOTHESES.md, Experiment B; results/expB_ollama_offload_sweep.jsonl
Placement gains, Figure 3	README.md validated-results table; results/exp_qwen38_27b_iq2xxs_task.jsonl for the 27B row
Context sweep, Figure 4	PRODUCT_HYPOTHESES.md, Experiment C; results/expC_context_sweep.jsonl and expC_context_sweep_summary.json
Scheduler ablation, Figure 5	PRODUCT_HYPOTHESES.md, the four-way ablation; results/exp_scheduler_ablation.jsonl
End-to-end agent A/B, Figure 6	PRODUCT_HYPOTHESES.md, the pi-agent validation arc and its closing synthesis
Tiering strategies, Figure 7	docs/ARCHITECTURE.md, Phases 3 to 5; results/20260902_171830_phase5_cache_test
Context-key and headroom failures, Section 7.3	PRODUCT_HYPOTHESES.md, context-aware placement section; pick_best in scripts/ramer_local.py
MoE unreachability, Section 9.3	PRODUCT_HYPOTHESES.md, MoE expert placement section

To re-measure a placement on different hardware:

```
python -m app.cli doctor
python -m app.cli benchmark <model> --num-ctx 8192
python -m app.cli serve --port 11435
```

The figures themselves are regenerated with `python tools/survey_figures.py` and this document with `python tools/build_survey_pdf.py`.